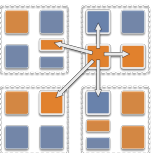


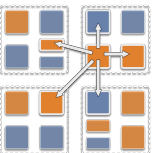
Dynamic Load Balancing

- Object-based decomposition allows the runtime system to remap objects dynamically to balance load across processors.
 - How does the RTS decide where to map objects?
 - Different load balancing strategies are supported in Charm4Py.
 - Most naively, the RTS could just move objects away from overloaded processors to underloaded processors. This is called “GreedyLB.”
- How is load determined?



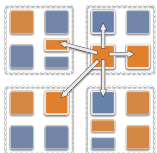
Measurement Based Load Balancing

- Principle of Persistence
 - Object communication patterns and computational loads tend to persist over time
 - In spite of dynamic behavior
 - Abrupt but infrequent changes
 - Slow and small changes
 - Recent past is a good predictor of near future
- Runtime instrumentation
 - Measures communication volume and computation time
- Measurement-based load balancers
 - Measure load information for chares
 - Periodically use the instrumented database to make new decisions and migrate objects
 - Many alternative strategies can use the database



Using the Load Balancer

- Link a LB module
 - `-module <strategy>`
 - RefineLB, NeighborLB, GreedyCommLB, others
 - EveryLB will include all load balancing strategies
- Compile time option (specify default balancer)
 - `-balancer RefineLB`
- Runtime option (override default)
 - `+balancer RefineLB`



Using the Load Balancer

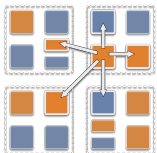
Enable LB at runtime:

a. Using the option **+balancer**

Eg: **python charmrun.start +p4 example.py <args> +balancer <name of LB>**

b. Some supported load balancing strategies include:

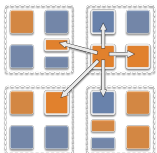
- i. GreedyLB: iteratively assigns the heaviest object to the least loaded processor.
- ii. GreedyRefineLB: iteratively assigns the heaviest object on the most overloaded processor to the least loaded processor.



Using the Load Balancer

Instrument your code:

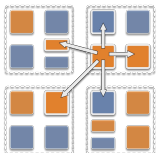
- a) the provided **AtSync()** call acts as a *non-blocking* barrier. When all participating objects call **AtSync**, the RTS begins load balancing.
- b) “Participating objects” are distinguished by setting **usesAtSync=true** in the constructor.
- c) Once all local migrations are completed, the load balancer calls the virtual function **ResumeFromSync()** on each of the participating objects.
- d) The Charm4py developer should implement **ResumeFromSync()**.



Example: Charm4py Particle Simulation

- simple simulation of particles moving randomly in a 2D space
- chares with more particles do more work
- load balancing is called every 20 simulated iterations
- this example can be found in [charm4py/examples/particle](https://charm4py.org/examples/particle)

```
def main(args):  
    ...  
    # chare array is constructed with useAtSync=True  
    cells = Array(Cell, array_dims, args=[...], useAtSync=True)  
    cells.run() # begin simulation iterations
```

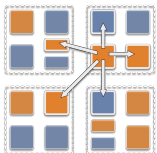


Example: Charm4py Particle Simulation

```
def run(self):
    if self.iteration == NUM_ITER:
        return
    # do iteration work here
    self.iteration += 1
    if self.iteration % 20 == 0: # cells call AtSync every 20 iters
        # load balancing will start when all the cells call this:
        self.AtSync()
        return # now exit because AtSync is non-blocking
    self.run()
```

note: the charm4py repo demonstrates a different variation on this control structure, simplified here

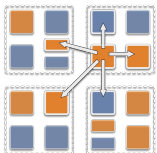
```
# this is called by the runtime when load balancing finishes
def resumeFromSync(self):
    # resume the simulation
    self.thisProxy[self.thisIndex].run() # start next iter
```



Instrumentation

- By default, instrumentation is enabled
 - Automatically collects load information
- Sometimes, you want LB decisions to be based only on a portion of your program
 - To disable by default, provide runtime argument `+L 0-f`
 - To toggle instrumentation in code, use `LBTurnInstrumentOn()` and `LBTurnInstrumentOff()`

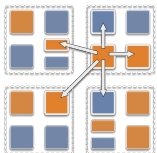
CHARM++ SLIDE



Code to Use Load Balancing

- Write PUP method to serialize the state of a chare
- Set `usesAtSync = true;` in chare constructor
- Insert `AtSync()` call at a natural barrier
 - Call from every chare in all collections
 - Does not block
- Implement `ResumeFromSync()` to resume execution
 - A typical `ResumeFromSync()` contributes to a reduction

CHARM++ SLIDE



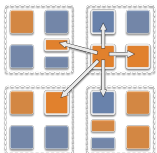
Example: Stencil

```
// Synchronize at every iteration: Main starts next iteration
void Main::endIter() { stencilProxy.sendBoundaries(); }

// Assume a 1D Stencil chare array with near neighbor communication
void Stencil::sendBoundaries() {
    thisProxy(wrap(x-1)).updateGhost(RIGHT, left_ghost);
    thisProxy(wrap(x+1)).updateGhost(LEFT, right_ghost);
}

void Stencil::updateGhost(int dir, double ghost) {
    updateBoundary(dir, ghost);
    if (++remoteCount == 2) {
        remoteCount = 0;
        dowork(); } }
}
```

CHARM++ SLIDE



Example: Stencil cont.

```
void Stencil::doWork() {  
    underThreshold = (computeKernel() < DELTA);  
    contribute(CkCell)ack(CkReductionTarget(Main, endIter), ai Proxy);  
}
```

CHARM++ SLIDE

